



NVIDIA TensorRT Optimization

Accelerating Inference on NVIDIA GPUs

A comprehensive guide for ML and infrastructure engineers to maximize GPU inference performance

Why TensorRT?

TensorRT is NVIDIA's specialized SDK designed specifically for **high-performance inference** on NVIDIA hardware, delivering:

Lower Latency

Reduce inference time by up to 8x compared to framework defaults

Higher Throughput

Process more inferences per second on the same hardware

Reduced Cost

Optimize GPU utilization for better ROI in production environments



Widely used in LLMs, computer vision,
and edge AI deployments

Core Optimizations

1

Layer Fusion

Combines multiple operations into fewer GPU kernels, reducing memory transfers and kernel launch overhead

Example: Conv + Bias + ReLU → single fused operation

2

Precision Calibration

Automatically converts FP32 operations to lower precision (FP16/INT8) with minimal accuracy loss through quantization

Can reduce memory bandwidth requirements by 2-4x

3

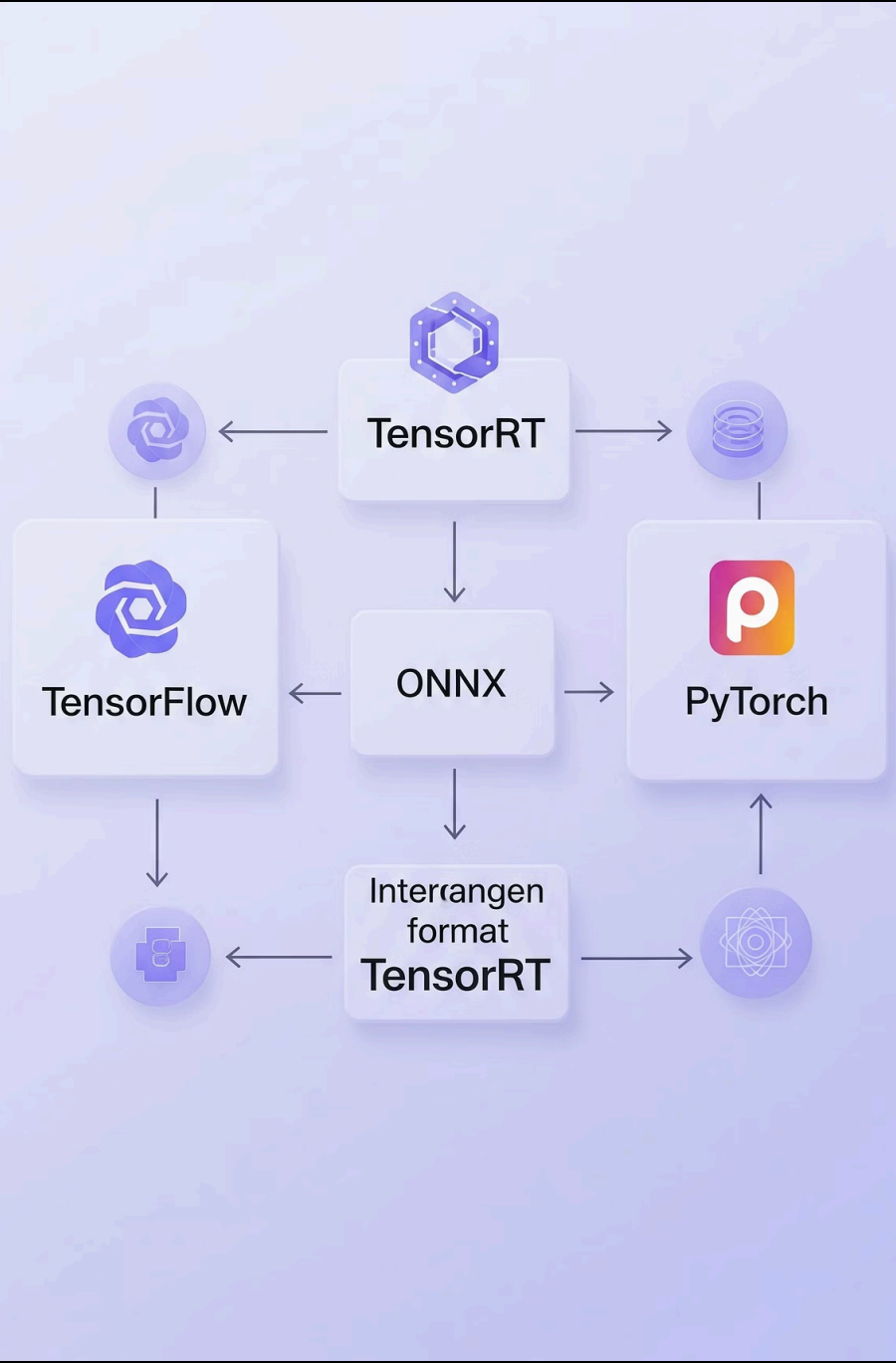
Kernel Auto-Tuning

Benchmarks multiple kernel implementations at build time and selects the fastest for your specific GPU architecture

4

Memory Optimizations

Reduces activation memory footprint and improves weight layout for better cache utilization and higher GPU occupancy



Supported Model Formats



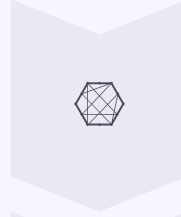
TensorFlow

Export SavedModel or convert to UFF format



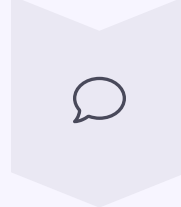
PyTorch

Use Torch-TensorRT or export to ONNX



ONNX

Universal exchange format with widest compatibility



TensorRT Engine

Optimized binary for deployment

```
trtexec --onnx=model.onnx --saveEngine=model.engine
```

Precision Modes

TensorRT supports multiple precision modes, enabling a balance between performance and accuracy:

1x

FP32 (Full Precision)

Baseline performance with maximum accuracy

2x

FP16 (Half Precision)

2x smaller memory footprint with near-identical accuracy

4x

INT8 (8-bit Integer)

4x smaller footprint, highest performance, requires calibration dataset

6x

FP8 (Emerging)

Available on Hopper GPUs, further memory savings



TensorRT Workflow Overview

Export Model

Convert trained model to ONNX or TorchScript format

Example: `torch.onnx.export(model, sample_input, "model.onnx")`

Optimize with TensorRT

Use TensorRT API or command-line tools to build an optimized engine

Select Precision

Choose FP16/INT8 based on accuracy/performance requirements

Deploy Engine

Serve via NVIDIA Triton or integrate into custom application

Benchmark Performance

Measure p95/p99 latency and throughput vs. baseline

Torch-TensorRT Integration

For PyTorch users, Torch-TensorRT provides a seamless integration path:

```
import torch_tensorrt

trt_model = torch_tensorrt.compile(
    model,
    inputs=[torch_tensorrt.Input((1,3,224,224))],
    enabled_precisions={torch.half}
)

# Use just like a regular PyTorch model
outputs = trt_model(inputs)
```

- Drop-in replacement for your PyTorch model
- Use `torch.jit.trace` or `torch.jit.script` first
- Supports dynamic shapes and custom plugins



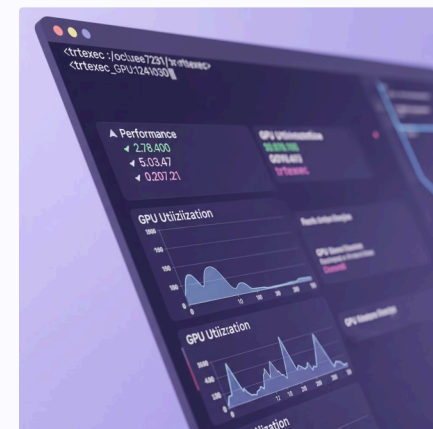
Using trtexec Command-Line Tool

The `trtexec` utility enables quick model conversion and benchmarking in one step:

```
trtexec --onnx=model.onnx \  
  --fp16 \  
  --shapes=input:1x3x224x224 \  
  --saveEngine=model.engine \  
  --verbose
```

Key Features:

- Reports latency, throughput, and GPU memory usage
- Tests different batch sizes with `--shapes`
- Supports precision modes (`--fp16`, `--int8`)
- Enables benchmarking without writing code



Deployment Options



Standalone TensorRT

Direct integration via C++/Python bindings

Full control over execution and memory



NVIDIA Triton Server

Multi-model serving with dynamic batching

Built-in metrics and scaling capabilities



Kubernetes Integration

Container orchestration with autoscaling

Cloud-native monitoring and deployment



Edge Deployment

Jetson devices with TensorRT runtime

Optimized for power-constrained environments

Performance Optimization Tips

Static vs. Dynamic Shapes

Use **static shapes** when possible - dynamic shapes add overhead but provide flexibility when input dimensions vary

Engine Warm-up

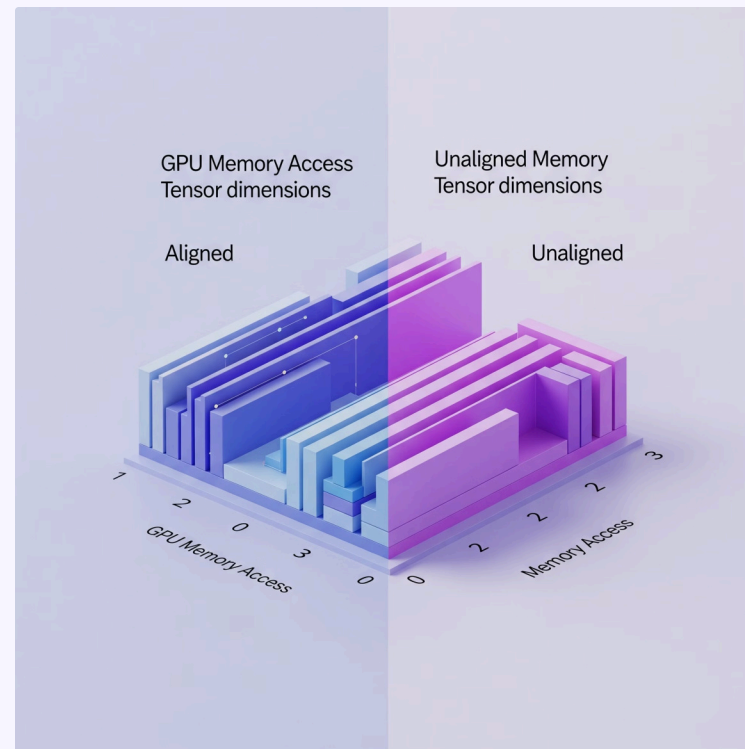
Always run several **warm-up inferences** before production load to populate GPU caches

GPU-Friendly Dimensions

Align input shapes with **multiples of 8/16/32** to maximize tensor core utilization

Performance Metrics

Monitor both **p95/p99 latency** and **throughput** - optimize for your specific use case constraints



GPU memory access efficiency with aligned vs. unaligned tensors

Observability & Debugging

TensorRT Profiling Tools

- **TensorRT Verbose Logs:** View kernel fusion decisions and precision choices
- **Nsight Systems:** Timeline-based profiling of GPU activities
- **Nsight Compute:** Detailed kernel-level performance analysis

Key Metrics to Monitor

- **SM Utilization:** How effectively are GPU cores used?
- **Memory Throughput:** Is the model bandwidth-limited?
- **Kernel Duration:** Identify bottleneck operations
- **Accuracy Comparison:** Validate against FP32 baseline



Nsight Systems timeline view helps identify bottlenecks in your inference pipeline

Common Pitfalls & Solutions

Unsupported Operations

Problem: Exotic ONNX ops cause fallback to framework runtime

Solution: Implement custom plugins or replace with supported alternatives

Calibration Issues

Problem: Poor INT8 accuracy due to inadequate calibration dataset

Solution: Use representative dataset with 100+ samples covering the input distribution

Precision Problems

Problem: Over-aggressive precision causing unstable predictions

Solution: Set precision per-layer with TensorRT API for sensitive operations

Batch Size Neglect

Problem: Default batch size leads to underutilized GPU

Solution: Profile different batch sizes to find optimal throughput/latency balance

Most TensorRT issues can be resolved with proper configuration and understanding the optimization process

Key Takeaways

Best-In-Class GPU Optimization

TensorRT delivers unmatched inference performance on NVIDIA GPUs through sophisticated kernel optimizations

Precision-Driven Performance

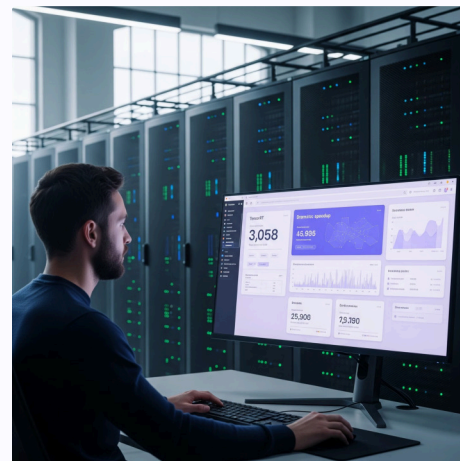
FP16/INT8 precision modes deliver 2-4x performance gains with minimal accuracy loss

Framework Compatibility

Works seamlessly with [ONNX](#), [PyTorch](#), and [TensorFlow](#) models

Production-Ready Deployment

Deploy via [Triton Inference Server](#) for scalable, observable production serving



Next Steps:

- Benchmark your models with TensorRT
- Explore precision tuning options
- Integrate with your deployment pipeline